23MID0018

RAJAVELMURUGAN S

1. Investigate the Dataset and document your findings.

2. Perform necessary Pre-Processing, add any column as required for your analysis, train a

model for predicting Loan Eligibility, save the model file and test its accuracy.

3. Report the accuracy and take steps to improve further on accuracy.

4. Document the measures that you take to prevent overfitting.

In [58]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

In [59]:
```
df = pd.read_csv("/home/matlab/Loan Eligibility Prediction.csv") # reading dataset
print(df.head())
print(df.info())
print(df.isnull().sum())
```

```
   Customer_ID  Gender Married  Dependents     Education Self_Employed  \
0          569  Female      No           0      Graduate            No
1           15    Male     Yes           2      Graduate            No
2           95    Male      No           0  Not Graduate            No
3          134    Male     Yes           0      Graduate           Yes
4          556    Male     Yes           1      Graduate            No

   Applicant_Income  Coapplicant_Income  Loan_Amount  Loan_Amount_Term  \
0              2378                 0.0            9               360
1              1299              1086.0           17               120
2              3620                 0.0           25               120
3              3459                 0.0           25               120
4              5468              1032.0           26               360

   Credit_History Property_Area Loan_Status
0               1         Urban           N
1               1         Urban           Y
2               1     Semiurban           Y
3               1     Semiurban           Y
4               1     Semiurban           Y
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Customer_ID         614 non-null    int64
 1   Gender              614 non-null    object
 2   Married             614 non-null    object
 3   Dependents          614 non-null    int64
 4   Education           614 non-null    object
 5   Self_Employed       614 non-null    object
 6   Applicant_Income    614 non-null    int64
 7   Coapplicant_Income  614 non-null    float64
 8   Loan_Amount         614 non-null    int64
 9   Loan_Amount_Term    614 non-null    int64
 10  Credit_History      614 non-null    int64
 11  Property_Area       614 non-null    object
 12  Loan_Status         614 non-null    object
dtypes: float64(1), int64(6), object(6)
memory usage: 62.5+ KB
None
```

```
Customer_ID           0
Gender                0
Married               0
Dependents            0
Education             0
Self_Employed         0
Applicant_Income      0
Coapplicant_Income    0
Loan_Amount           0
Loan_Amount_Term      0
Credit_History        0
Property_Area         0
Loan_Status           0
dtype: int64
```

In [60]:
```
target = "Loan_Status" # set the target as Loan_Status
df = df.drop_duplicates() # Drop duplicates in datasets
df = df.dropna(subset=[target])
df = df.drop(columns=["Customer_ID"], errors="ignore")
X = df.drop(columns=[target]) # for training except the target we need to give for training to X
y = df[target] # We need to give Loan_Status as target attribute to Y
```

In [70]:
```
num_cols = X.select_dtypes(include=np.number).columns.tolist()   # splitting the numerical colums to num_cols
cat_cols = X.select_dtypes(exclude=np.number).columns.tolist()  # splitting he categorical columns to cat_cols

numeric_pipe = Pipeline([
("imputer", SimpleImputer(strategy="median")), # impute the values using SimpleImputer
("scaler", StandardScaler()) # scale the values using Standardscaler
])

categorical_pipe = Pipeline([
("imputer", SimpleImputer(strategy="most_frequent")), # impute the values using SimpleImputer
("encoder", OneHotEncoder(handle_unknown="ignore")) # encode the values using OneHotEncoder
])

preprocessor = ColumnTransformer([          # Using columnTransformer to combine the numerical and cat values
("num", numeric_pipe, num_cols),
("cat", categorical_pipe, cat_cols)
])
X_train, X_test, y_train, y_test = train_test_split( # split the data into train and test 80% 20%
```

```
X, y, test_size=0.2, random_state=42
)

model = Pipeline([
("preprocessor", preprocessor),
("classifier", LogisticRegression(max_iter=1000)) # Classifier :Logistic Regression
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred)) # Accuracy of the trained model
```

```
Accuracy: 0.8211382113821138
```

In [62]:
```
print("Confusion Matrix:\n", confusion_matrix(y_test, pred)) #Confusion matrix
print("Classification_Report:\n",classification_report(y_test, pred)) # Classification report
```

```
Confusion Matrix:
 [[19 21]
 [ 1 82]]
Classification_Report:
               precision    recall  f1-score   support

           N       0.95      0.47      0.63        40
           Y       0.80      0.99      0.88        83

    accuracy                           0.82       123
   macro avg       0.87      0.73      0.76       123
weighted avg       0.85      0.82      0.80       123
```

In [66]:
```
print("\nEnter new applicant details:\n")
applicant = {}

for col in X.columns:
    value = input(f"{col} ({X[col].dtype}): ")
if col in num_cols:
    applicant[col] = pd.to_numeric(value, errors="raise")
else:
    applicant[col] = value
```

```
new_data = pd.DataFrame([applicant], columns=X.columns)
prediction = model.predict(new_data)[0]

print("\nPredicted class:", prediction)
print("Predicted loan status:", prediction)
```

```
Enter new applicant details:

Predicted class: Y
Predicted loan status: Y
```

In [ ]: